

Programme: Le jeu du nombre mystère

Objectifs

Dans cet exercice, vous allez revoir quelques notions ou apprendre à :

- utiliser la fonction **input()**
- convertir une valeur avec **int()**
- utiliser une boucle **while**
- utiliser des conditions (**if / elif / else**)
- utiliser un nombre **aléatoire**
- **compter** le nombre de tentatives

Règle du jeu

Le ou la maître·sse du jeu choisit **un nombre entier entre 1 et 100**. Ce nombre est appelé **le nombre mystère**. Le ou la joueur·euse doit essayer de **deviner ce nombre**.

À chaque tentative, le ou la joueur·euse propose un nombre :

- si le nombre proposé est **trop grand**, le ou la maître·sse du jeu répond : “**Trop grand**” ;
- si le nombre proposé est **trop petit**, il ou elle répond : “**Trop petit**” ;
- si le nombre proposé est correct, le ou la joueur·euse trouve le **nombre mystère**.

Le but du jeu est de **trouver le nombre mystère en le moins d’essais possible**

Première étape: Programme de base - une seule partie

Commencer par ouvrir l’éditeur **Thonny**. Enregistrez **tout de suite** votre programme dans votre dossier **Documents\...** Le fichier doit être nommé correctement avec votre nom: “**votre_nom**”_devine_nombre.py

Comment commencer ?

1. Commencer par créer le cartouche d’en-tête comme dans le projet dessin (nom, date, ...).
2. L’ordinateur choisit au hasard un nombre entre 1 et 100.
3. Le programme demande (input) au·à la joueur·se d’entrer un nombre.

4. À chaque tentative, l'ordinateur indique si le nombre mystère est plus grand ou plus petit que le nombre proposé.
5. Tant que le nombre n'est pas trouvé, on recommence à l'étape 3.

Prenons un exemple où l'ordinateur a tiré 72 comme nombre aléatoire (mais vous ne le savez pas):

```
Bienvenue dans le jeu du nombre mystère
Voilà, j'ai choisi un nombre entre 1 et 100, à toi de jouer!

Entrez un nombre: 35
Plus grand...
Entrez un nombre: 50
Plus grand...
Entrez un nombre: 86
Plus petit...
Entrez un nombre: 75
Plus petit...
Entrez un nombre: 70
Plus grand...
Entrez un nombre: 72
C'est gagné !
```

Pour utiliser la fonction `randint` qui retourne un nombre entier au hasard, vous devez importer son module¹ de la manière suivante dans les premières lignes de votre code :

```
from random import randint
```

Une fois importé, vous pouvez utiliser la fonction de cette manière :

```
n = randint(minimum,maximum)
```

La variable `nombre_entier_hasard` contiendra un nombre entier compris en 1 et 100.

¹ **Rappel:** On appelle "module" tout fichier constitué de code Python (c'est-à-dire tout fichier avec l'extension `.py`) importé dans un autre fichier ou script. Les modules permettent la séparation et donc une meilleure organisation du code.

Deuxième étape

Complétez le programme de l'exercice précédent en ajoutant les **options suivantes** et en validant chaque étape:

- A. Ajoutez un compteur qui compte le nombre de tentatives.

À la fin de la partie, le programme affiche ce nombre et un commentaire si :

Le-la joueur·euse a trouvé du premier coup:

```
Vous avez gagné en 1 coup(s) ...  
Excellent !
```

Le-la joueur·euse a trouvé en moins de 5 coups:

```
Vous avez gagné en 4 coup(s) ...  
Pas mal !
```

Sinon:

```
Vous avez gagné en 7 coup(s) ...  
Mais vous pouvez mieux faire !
```

- B. Limitez le nombre de tentatives à 10. Si le-la joueur·euse n'a pas trouvé en 10 tentatives, le programme donne la réponse et se termine. Par exemple :

```
Vous avez perdu !  
Vous avez joué 10 fois sans trouver ...  
La réponse était 42.
```

- C. Nouvelle option:

Après 5 essais, le programme avertit qu'il ne reste plus que 5 tentatives:

```
ATTENTION: Il vous reste encore 5 tentatives.
```

- D. À la fin de la partie, le programme propose de rejouer.

Si la lettre O est saisie le jeu recommence sinon il termine.

```
Voulez-vous rejouer? [O/N]:
```

- E. Afficher un complément de message si le-la joueur·euse est très proche du nombre mystère (par exemple à moins de 5).

NB: Vous pouvez utiliser la fonction intégrée `abs()` qui retourne la valeur absolue (non négative) d'un nombre. Ex: `abs(nombre_entier_hasard - nombre)`

Par exemple, si le nombre mystère est 44:

```
Bienvenue dans le jeu du nombre mystère
Voilà, j'ai choisi un nombre entre 1 et 100, à toi de jouer!

Entrez un nombre: 35
Plus grand...
Entrez un nombre: 50
Plus petit...
Entrez un nombre: 40
Vous chauffez !
Plus petit...
Entrez un nombre: 44
C'est gagné !
```

- F. Est-ce que le nombre saisi est compris entre 1 et 100. Si ce n'est pas le cas demander à l'utilisateur·trice de saisir à nouveau un nombre.

```
Entrez un nombre: 103
Votre nombre est trop grand, veuillez recommencer !
Entrez un nombre:
```

- G. Factoriser votre code.

Créer une fonction `verifie_nombre` avec deux paramètres qui sont le nombre choisi au hasard et celui de l'utilisateur·trice. La fonction doit afficher le texte "Plus grand" ou "Plus petit" ou rien.

```
def verifie_nombre(nb_hasard, nb_saisi):
    ...
```

Troisième étape

Cette fois, c'est à l'ordinateur de trouver le nombre choisi.

Après avoir discuté en classe sur la solution optimale, implémenter cet algorithme de recherche du nombre dans votre code de la 1ère étape :

- On définit les bornes inférieures et supérieures dans lesquelles on cherche le nombre,
- plutôt que de demander un nombre à l'utilisatrice, votre programme calculera lui-même la prochaine valeur optimale,
- puis mettra à jour les bornes en fonction de si ce nombre est trop grand ou trop petit.